Phoenix Max Lookback and TTL Redesign

April 2023

Kadir Ozdemir


THANK YOU

Outline

- Background
  - HBase Cell Types
  - HBase Data Retention Policy
  - Phoenix SCN and Max Lookback
  - Empty Column
- Phoenix TTL and Max Lookback Issues
- Redesign
  - Requirements
  - Suggested Phoenix Data Retention Policy
  - TTL and Max Lookback Redesign
  - Phoenix Compaction
- Testing Strategy
- Next Steps

HBase Cell Types

Row Key	Column Family	Column Qualifier	Timestamp	**Type**	MVCC	Value

1. **Put**: stores data
2. **Delete Marker**
   a. **Delete[Column Version]**: Deletes the current cell version of its row and column.
   b. **Delete Column**: Deletes all cell versions of its row and column
   c. **Delete Column Family Version**: Deletes the current cell versions (the current cell version of each column) within its row and column family
   d. **Delete Column Family**: Deletes all cell versions within its row and column family

HBase Data Retention Policy

1. **VERSIONS**: The maximum number of cell versions to retain
2. **MIN_VERSIONS**: The minimum number of cell versions to retain
3. **TTL**: Time to live for cells
4. **KEEP_DELETED_CELLS**
   a. **FALSE** : Remove the delete markers and the delete cells
   b. **TRUE** : Retain at most VERSIONS live (not expired) delete markers and deleted cell versions
   c. **TTL** : Retain live delete markers and deleted cells until the delete markers expires

SYSTEM CHANGE NUMBER (SCN)

- In HBase and Phoenix, the current System Change Number (SCN) is the current timestamp (or wall clock time)
- Phoenix allows changing SCN for a given Phoenix connection to go back in time and retrieve the latest version of cells at that time
- The cluster level max lookback age defines the lookback time window size
- This means
  - Row versions that are visible through the max lookback window should be retained
  - Row versions that are not visible through the max lookback window are not needed

Empty Column

- A Phoenix table may not have any non-PK columns
- To form a cell, Phoenix needs to have a non-PK column
- **Empty column**: An internal non-PK column that is not visible to users
- Phoenix includes an empty column cell in every mutation

Data Integrity Issues

Partial Row Retention

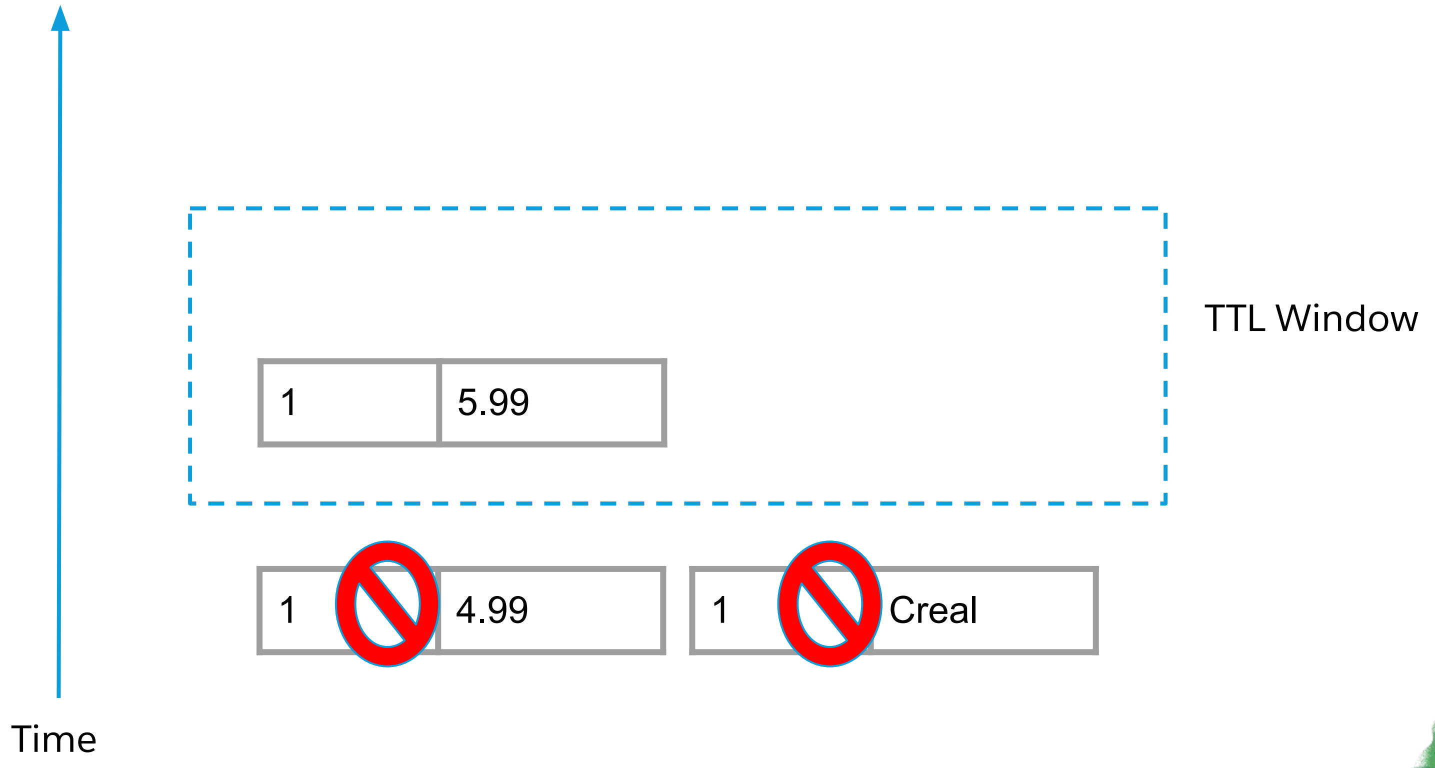
HBase data retention policies are executed at the cell level and cause partial row retention

ID	Price	Description
1	5.99	

ID	Price	Description
1	5.99	Creal

ID	Price	Description
1	4.99	Creal

VERSIONS=1, MIN_VERSIONS=0,
KEEP_DELETED_CELLS = FALSE



Storage Waste and Data Loss Issues

Max Lookback Issue

- HBase provides one data retention time window: **TTL**
- Phoenix needs two windows: **Max Lookback** and **TTL**
- The max lookback age is extended to the TTL at run time
- In the max lookback window, all cell versions and delete markers have to be retained
- **Storage Waste**: Deleted data cannot be removed until it expires
- **Data Loss**: When the data is inserted back, it is masked by delete markers

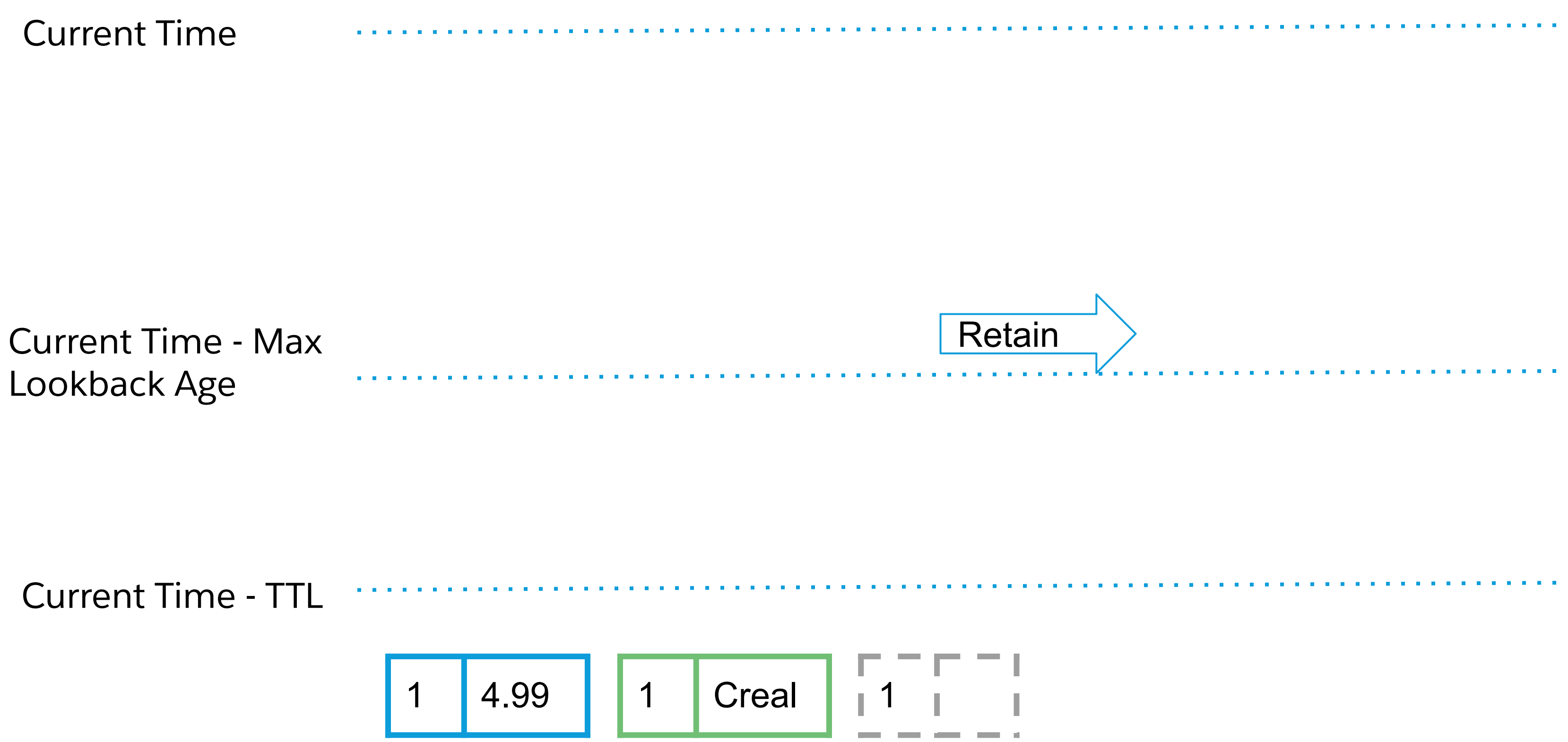
Requirements for Redesign

- Row versions should not expire partially
- Live/deleted row versions and delete markers visible through the max lookback window should be retained
- Live, expired or deleted row versions not visible through the max lookback window should not be retained (unless they are retained by the HBase data retention policy)
- The expired mutations should be masked
- The solution that meets the above requirements should not degrade the performance, scalability or availability of Phoenix

Phoenix Data Retention Policy

Proposed

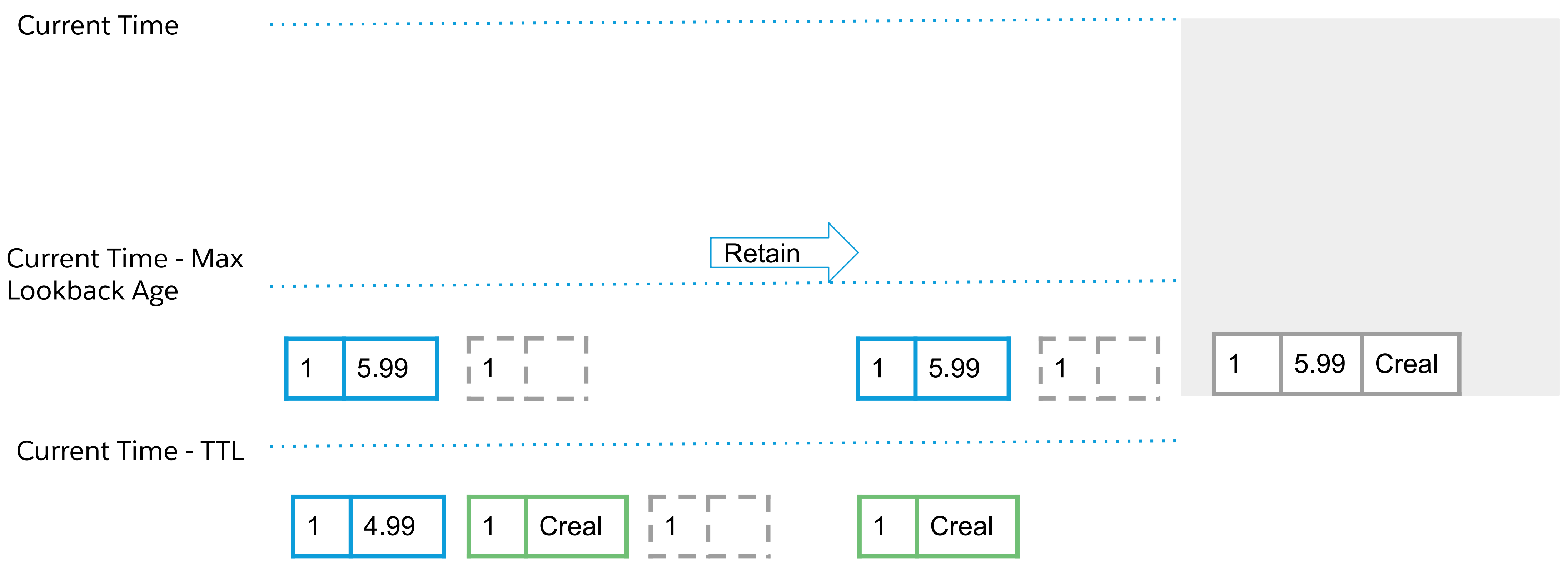
- Retain all row versions and delete markers visible through the max lookback window
- Retain additional cells required by the configured HBase data retention policy if needed



Phoenix Data Retention Policy

Proposed

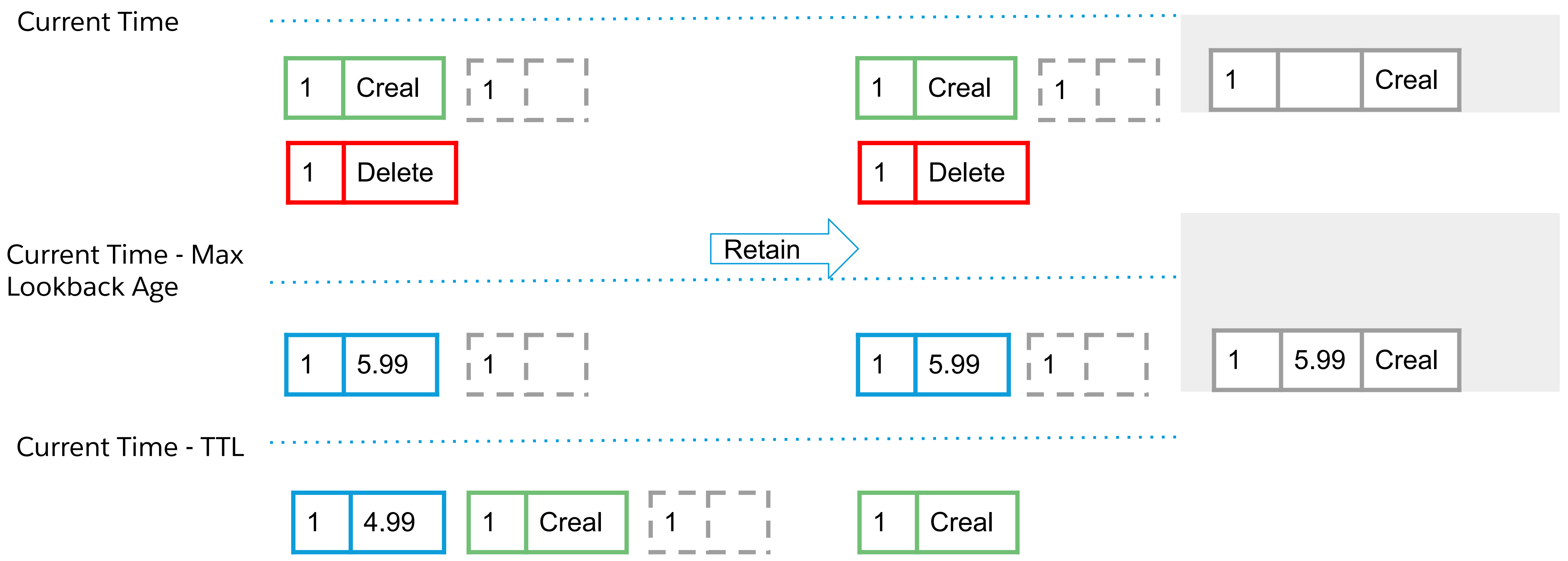
- Retain all row versions and delete markers visible through the max lookback window
- Retain additional cells required by the configured HBase data retention policy if needed



Phoenix Data Retention Policy

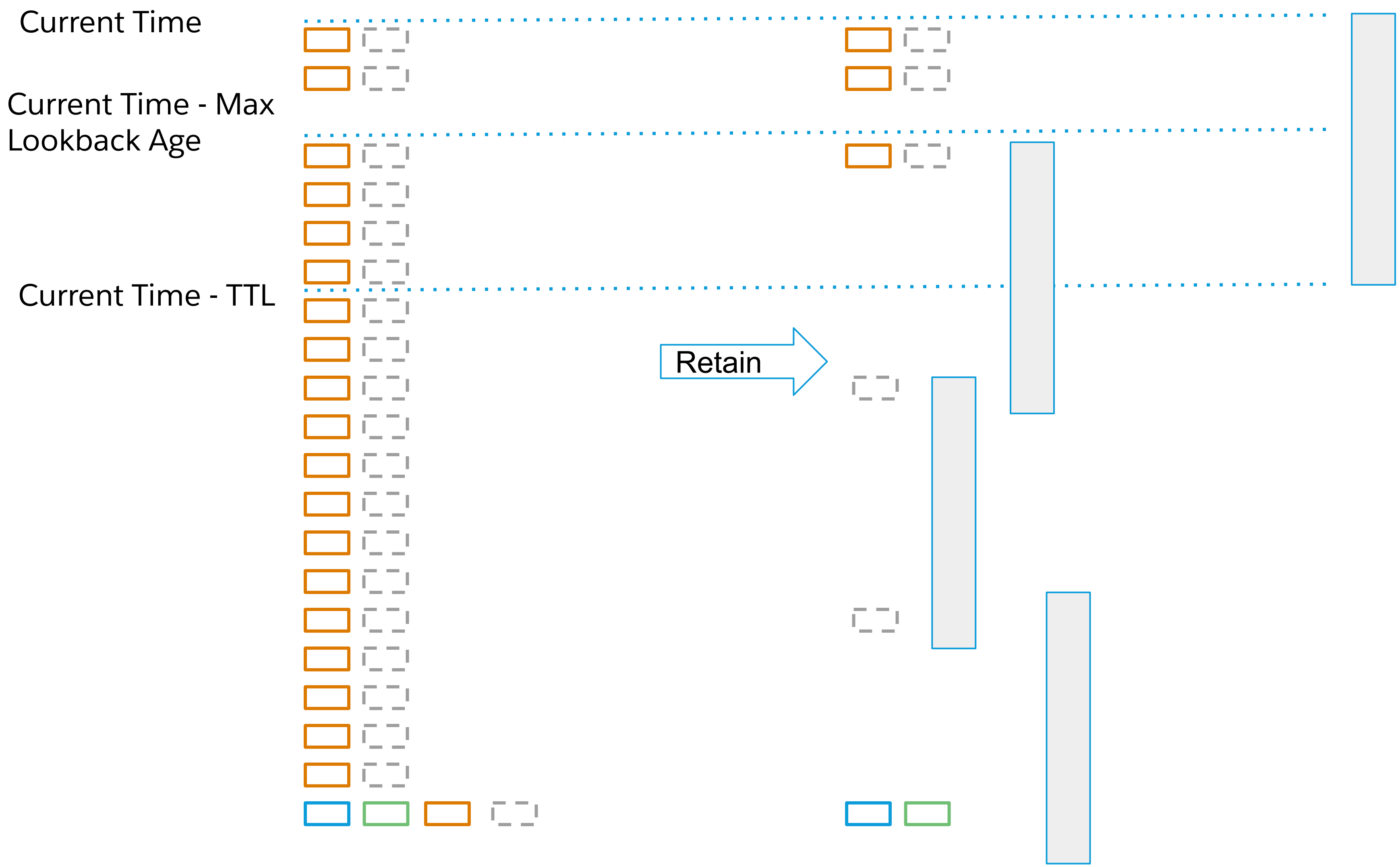
Proposed

- Retain all row versions and delete markers visible through the max lookback window
- Retain additional cells required by the configured HBase data retention policy if needed

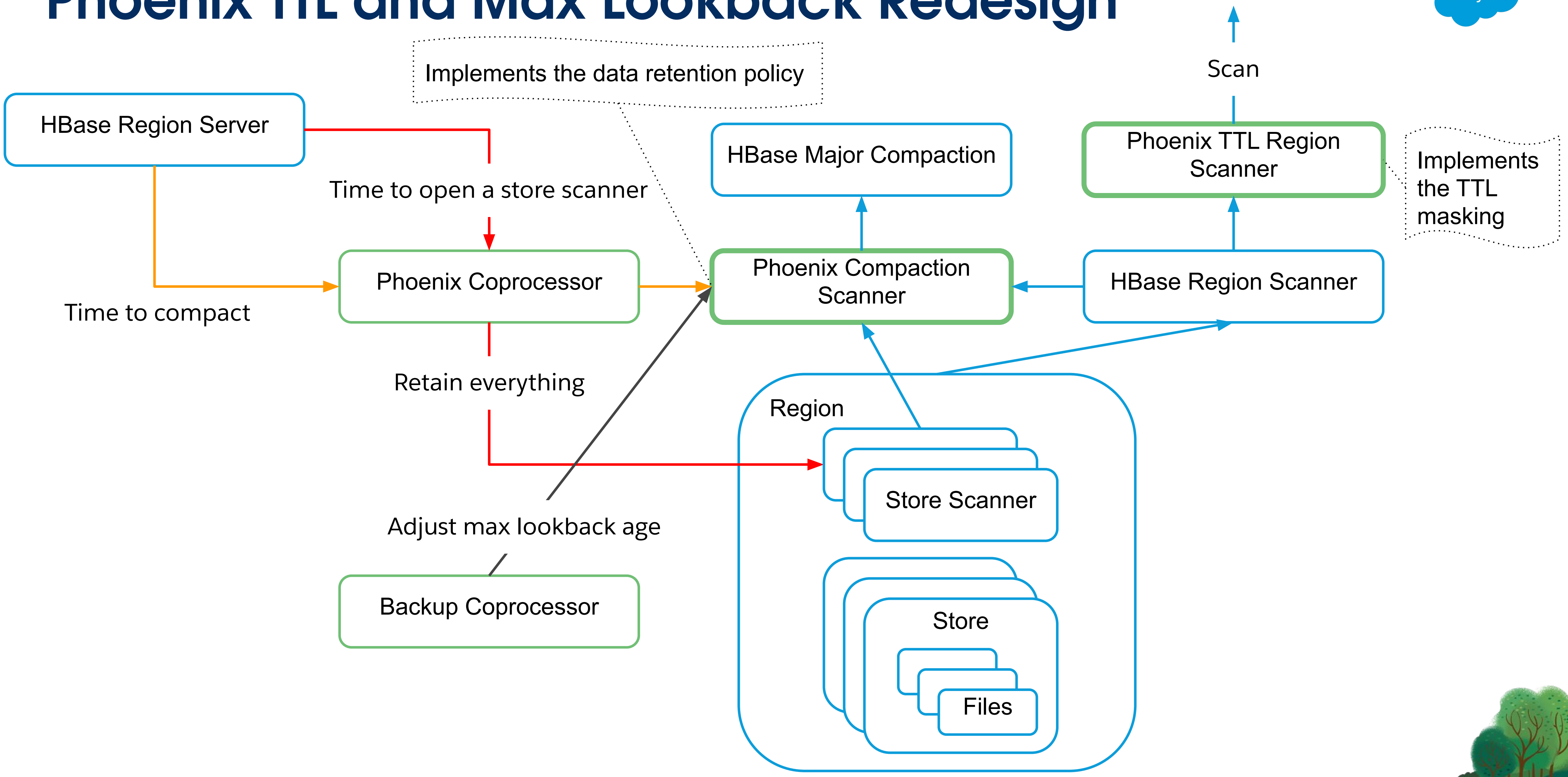


Phoenix Data Retention Policy

Proposed

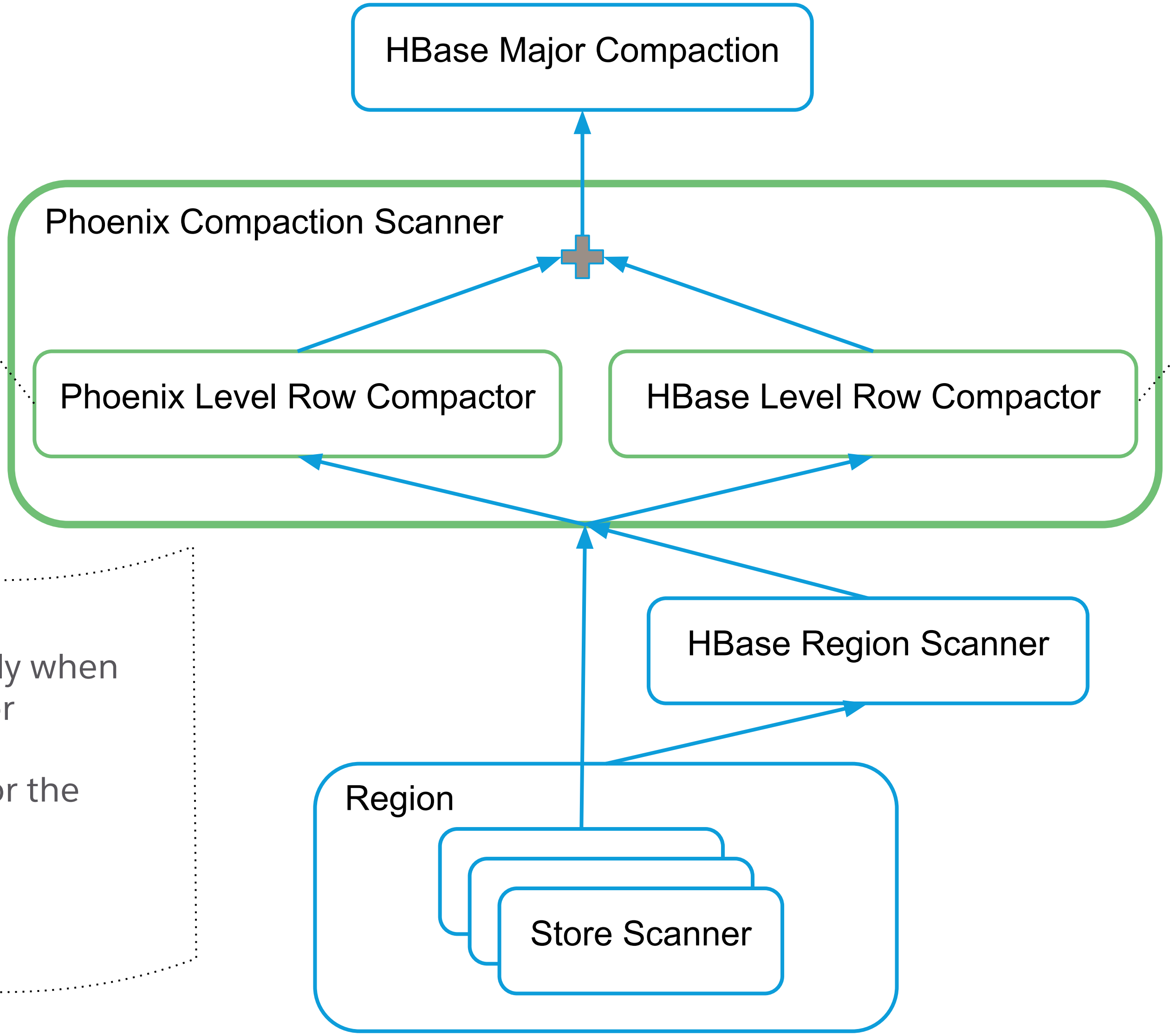


Phoenix TTL and Max Lookback Redesign



Phoenix Compaction Scanner

Retain all row versions and delete markers visible through the max lookback window



Implements the HBase data retention policy

- HBase Level Row Compactor is used only when VERSIONS > 1 or MIN_VERSIONS > 0, or KEEP_DELETED_CELLS != FALSE
- Compaction is done at the store level for the tables with one column family

Phoenix Row Level Compactor

- Sort the cells of a row based on their timestamps in descending order
- Whenever the time gap between subsequent cells is larger than TTL, trim the rest of cells
- Retain all the cells whose age is less than max lookback age
- Retain the cells of the last row version visible via the max lookback window
- After ordering the cells of the last row version based on their timestamps, check the time gap between cells
- For the gaps that are larger than TTL, retain a minimum number of empty column cells to make the gaps less than or equal to TTL

HBase Row Level Compactor

- A **compaction row version** (CRV) includes the latest (put) cell versions from each column such that the cell versions do not cross delete markers
- After forming a CRV, the next CRV is formed from the remaining cell versions
- HBase data retention rules are applied to CRVs (not to its put cells) and delete markers
- Note CRV is to prevent partial row expiration

Testing Strategy

- Create two tables with the same schema
- The same row is updated in a loop on both tables with the same content
  - Each update changes one or more columns chosen randomly with randomly generated values
  - The null value is also one of the generated values
  - Rows are deleted at random times
  - All rows are expired at random times
- Only compact the first table at random times
- After every upsert within the loop, compare two tables
  - All versions of the row visible to the maxlook window are compared
- Test with various TTL parameters and single and multiple column families

Next Steps

- Deploy it in 246
- Migrate to the Phoenix TTL table attribute
  - Remove the HBase TTL attribute
  - Required for supporting short TTL (7 days) and still preserving row integrity
- Build View TTL
- Build Tenant TTL
- Possibly leverage compaction for some background jobs

References

- Phoenix Table TTL and Max Lookback Redesign